

Registro de Prompts

AI Quality: O Desafio Gilded Rose - Go + ChatGPT

Este documento registra os prompts utilizados para gerar a suíte de testes do Gilded Rose em Go com o ChatGPT. Para cada uma das cinco abordagens testadas (prompt direto, Chain-of-Thought, Persona Pattern, Few-shot e Spec-driven), foi aberta uma conversa nova no ChatGPT, sem reaproveitar o contexto de um prompt para o outro, isso garante que a comparação entre as técnicas seja justa, já que cada uma parte do mesmo ponto de partida. Para cada prompt estão registrados o texto exato utilizado, o link da conversa com o resultado gerado pela IA e as observações feitas durante a análise da qualidade dos testes produzidos.

Código utilizado

O código abaixo foi copiado e colado em todos os trechos marcados como "[COLE AQUI O CÓDIGO]" ao longo deste documento.

Fonte: <https://github.com/emilybache/GildedRose-Refactoring-Kata/blob/main/go/gildedrose/gildedrose.go>

gildedrose/gildedrose.go

```
package gildedrose

type Item struct {
    Name      string
    SellIn, Quality int
}

func UpdateQuality(items []*Item) {
    for i := 0; i < len(items); i++ {

        if items[i].Name != "Aged Brie" && items[i].Name != "Backstage passes to a TAFKAL80ETC concert" {
            if items[i].Quality > 0 {
                if items[i].Name != "Sulfuras, Hand of Ragnaros" {
                    items[i].Quality = items[i].Quality - 1
                }
            }
        } else {
            if items[i].Quality < 50 {
                items[i].Quality = items[i].Quality + 1
                if items[i].Name == "Backstage passes to a TAFKAL80ETC concert" {
                    if items[i].SellIn < 11 {
                        if items[i].Quality < 50 {
                            items[i].Quality = items[i].Quality + 1
                        }
                    }
                }
                if items[i].SellIn < 6 {
                    if items[i].Quality < 50 {
                        items[i].Quality = items[i].Quality + 1
                    }
                }
            }
        }

        if items[i].Name != "Sulfuras, Hand of Ragnaros" {
```

```

        items[i].SellIn = items[i].SellIn - 1
    }

    if items[i].SellIn < 0 {
        if items[i].Name != "Aged Brie" {
            if items[i].Name != "Backstage passes to a TAFKAL80ETC concert" {
                if items[i].Quality > 0 {
                    if items[i].Name != "Sulfuras, Hand of Ragnaros" {
                        items[i].Quality = items[i].Quality - 1
                    }
                }
            } else {
                items[i].Quality = items[i].Quality - items[i].Quality
            }
        } else {
            if items[i].Quality < 50 {
                items[i].Quality = items[i].Quality + 1
            }
        }
    }
}
}

```

1. Direto (baseline)

Por que usar: *é o que a maioria faz no dia a dia, colar o código e pedir teste, sem técnica nenhuma, e serve como ponto de comparação.*

Prompt utilizado:

Aqui está o código Gilded Rose em Go:

[COLE AQUI O CÓDIGO]

Escreva testes unitários para esse código usando o pacote testing do Go.

Link da conversa com o resultado gerado:

<https://chatgpt.com/share/6a94728d-bedc-83e9-84e4-bb473ebe3ab9>

Observações:

Testes sólidos nos casos óbvios (4 tipos, limiares do Backstage Pass, dia do show), mas só testou valores já parados nos limites, não chegando até eles. Compilou sem problema.

2. Chain-of-Thought (CoT)

Por que usar: *força o modelo a narrar o raciocínio antes de codificar, o que tende a reduzir alucinação e aumentar a cobertura de casos de borda.*

Prompt utilizado:

Você vai gerar uma suíte de testes unitários em Go para o código abaixo, que implementa a lógica de atualização de qualidade de itens de uma loja (Gilded Rose).

Antes de escrever qualquer teste, siga este raciocínio passo a passo:

1. Liste todos os tipos de item presentes no código (normal, Aged Brie, Sulfuras, Backstage Passes, e qualquer outro que encontrar).

2. Para cada tipo, descreva as regras de negócio que você identificou no código: como Quality e SellIn mudam a cada dia, antes e depois da data de venda.
3. Identifique os limites (Quality nunca abaixo de 0, nunca acima de 50, exceto Sulfuras).
4. Liste os casos de borda que merecem um teste específico (ex: SellIn = 0, SellIn negativo, Quality = 0, Quality = 50, itens não documentados).
5. Só depois dessa análise, escreva os testes unitários em Go cobrindo cada caso identificado, usando o pacote testing (ou testify, se preferir).

Mostre seu raciocínio das etapas 1 a 4 antes de gerar o código de teste.

Código:

[COLE AQUI O CÓDIGO]

Link da conversa com o resultado gerado:

<https://chatgpt.com/share/6a947464-36f8-83e9-be5f-e2f5afc5655d>

Observações:

Melhorou visivelmente em relação ao baseline, testou a transição pro limite (ex: 49→50) e explicou o porque do comportamento no dia do vencimento. Compilou sem problema.

3. Persona Pattern

Por que usar: testa se dar uma identidade especialista ao modelo muda a profundidade da análise.

Prompt utilizado:

Você é um Engenheiro de Qualidade de Software Sênior, especialista em testar sistemas legados com lógica condicional complexa, com 15 anos de experiência revisando código crítico para bancos e varejistas. Você é extremamente cético em relação a testes superficiais e sempre busca os casos de borda que desenvolvedores juniores costumam esquecer.

Um desenvolvedor júnior te entregou o código Go abaixo, de um sistema de gerenciamento de estoque de uma loja (Gilded Rose), e pediu para você escrever a suíte de testes definitiva, que garanta que nenhuma regressão passe despercebida antes de ir para produção.

Escreva uma suíte de testes completa em Go, cobrindo comportamento normal e casos de borda, e explique brevemente, para cada grupo de testes, por que aquele caso é importante.

Código:

[COLE AQUI O CÓDIGO]

Link da conversa com o resultado gerado:

<https://chatgpt.com/share/6a947473-b36c-83e9-9ad6-7721ab2df927>

Observações:

Foi o mais rigoroso nas fronteiras (testou os dois lados de cada limiar) e trouxe um detalhe de Go que os outros não pegaram (nil vs. lista vazia). Compilou sem problema.

4. Few-shot (com exemplo)

Por que usar: verifica se dar um exemplo concreto de formato melhora a consistência e legibilidade dos testes gerados, sem precisar explicar regras.

Prompt utilizado:

Vou te mostrar um exemplo de como quero um teste unitário estruturado, e depois quero que você gere o restante seguindo esse mesmo padrão.

Exemplo:

```
func TestQualityDecreasesByOneForNormalItemBeforeSellDate(t *testing.T) {
    items := []*Item{{Name: "foo", SellIn: 10, Quality: 20}}
    gr := GildedRose{Items: items}
    gr.UpdateQuality()
    if items[0].Quality != 19 {
        t.Errorf("esperado 19, obtido %d", items[0].Quality)
    }
}
```

Agora, seguindo esse padrão de nomenclatura descritiva (TestQuando_Faz_O_Que) e essa estrutura Arrange-Act-Assert, gere a suíte de testes completa para o código abaixo, cobrindo Aged Brie, Sulfuras, Backstage Passes e itens normais, antes e depois da data de venda, e os limites de Quality (0 e 50).

Código:

[COLE AQUI O CÓDIGO]

Link da conversa com o resultado gerado:

<https://chatgpt.com/share/6a947493-9cac-83e9-85dc-e9cac4d10c1e>

Observações:

Não compilou. O erro que aparece primeiro é o typo `*testing.t` (deveria ser `*testing.T`), o Go para nele antes de chegar a avaliar o resto. Mas também usou a struct do exemplo (`GildedRose{Items: items}`) em vez da função real do código fornecido (`UpdateQuality(items []*Item)`), então mesmo corrigindo o typo ainda não compilaria. Boa cobertura de fronteiras, mas precisaria de correção manual pra rodar. Confirmado rodando `go test`.

5. Spec-driven (bônus)

Por que usar: testa uma variável diferente: em vez de a IA deduzir as regras lendo o código (como nos prompts 1 a 4), aqui as regras são dadas prontas. A comparação mostra se a qualidade dos testes depende mais da técnica de prompt ou de já ter a especificação explícita.

Prompt utilizado:

Aqui estão as regras de negócio do sistema Gilded Rose:

- Itens normais: Quality cai 1 por dia; cai 2 por dia depois que `SellIn < 0`; nunca fica negativa.
- "Aged Brie": Quality aumenta com o tempo, 2x mais rápido depois que `SellIn < 0`; nunca passa de 50.
- "Sulfuras, Hand of Ragnaros": nunca vendido, `SellIn` não muda, Quality sempre 80 e nunca muda.
- "Backstage passes": Quality sobe +1 normalmente, +2 se `SellIn <= 10`, +3 se `SellIn <= 5`; cai para 0 depois do show (`SellIn < 0`).
- Quality nunca é negativa e nunca passa de 50 (exceto Sulfuras).

Aqui está o código-fonte que implementa (ou deveria implementar) essas regras:

[COLE AQUI O CÓDIGO]

Gere testes unitários em Go que validem se o código realmente segue essas regras, incluindo casos que expõem possíveis violações.

Link da conversa com o resultado gerado:

<https://chatgpt.com/share/6a9474a0-0884-83e9-b2f0-abb85dbb8ec>

Observações:

Compilou bem, boa cobertura de fronteiras, mas o texto explicativo afirma que 3 testes 'deveriam falhar' quando na verdade eles passam.

6. Comparação entre as técnicas

A tabela abaixo resume as observações feitas em cada uma das cinco conversas, olhando para se o código gerado compilou, o quanto os testes cobriram casos de borda e a qualidade do raciocínio que o modelo apresentou junto com o código.

Técnica	Compilou?	Cobertura de casos de borda	Qualidade da explicação	Ponto forte	Ponto fraco
1. Direto	Sim	Testou os valores já nos limites, mas não a transição até eles	Nenhuma, só o código	Simplicidade, cobre os 4 tipos de item	Superficial, não prova que a lógica de borda funciona
2. Chain-of-Thought	Sim	Testou transições (ex.: 49 para 50)	Raciocínio explícito antes do código, explica o comportamento no dia do vencimento	Ganho real de profundidade sem precisar de exemplo ou spec externa	Não pegou o detalhe de Go (nil x lista vazia) que a Persona pegou
3. Persona Pattern	Sim	Mais rigorosa, testou os dois lados de cada limiar	Justificativa por grupo de teste	Único a identificar o caso nil x lista vazia	Nenhum relatado
4. Few-shot	Não	Boa cobertura de fronteiras, na teoria	Não se aplica, o problema é estrutural	Nomenclatura e padrão AAA consistentes	Typo (*testing.t) e uso da struct do exemplo em vez da função real do código
5. Spec-driven	Sim	Boa cobertura de fronteiras	Boa, mas com uma inconsistência	Regras de negócio prontas, sem precisar inferir do código	Disse que 3 testes deveriam falhar quando na verdade passam

Ranking geral de qualidade (do melhor para o pior, considerando corretude e profundidade):

1. Persona Pattern
2. Chain-of-Thought
3. Spec-driven
4. Direto (baseline)
5. Few-shot

7. Análise: o prompt fez diferença no resultado?

Comparando as cinco abordagens, dá para perceber que sim, o prompt fez diferença no resultado. As cinco conversas partiram do mesmo código, sem reaproveitar contexto entre elas, então a diferença de qualidade entre os testes gerados não é coincidência, ela vem da técnica usada em cada prompt.

O prompt Direto (baseline) mostra o que acontece quando não se guia o raciocínio do modelo. Sem nenhuma instrução sobre como pensar antes de responder, o ChatGPT foi direto para o código e gerou testes que cobrem os casos óbvios, mas testou só os valores que já estavam parados nos limites, sem testar a transição até eles. Isso mostra que, sem orientação, o modelo tende a fazer o mínimo necessário para entregar uma resposta que parece completa.

O Chain-of-Thought e o Persona Pattern mostram que pedir para o modelo pensar antes de codificar melhora bastante a cobertura de casos de borda. No CoT, como o prompt obriga o modelo a listar os tipos de item, as regras de negócio e os limites antes de escrever qualquer teste, ele já corrigiu o principal problema do baseline e passou a testar a transição (por exemplo, de 49 para 50), além de explicar o motivo do comportamento no dia do vencimento. A Persona foi ainda mais longe. Ao pedir que o modelo assumisse o papel de um engenheiro sênior cético com testes superficiais, ele testou os dois lados de cada limiar e ainda identificou um detalhe da linguagem Go, a diferença entre nil e uma lista vazia, que nenhuma outra técnica pegou. Isso sugere que colocar o modelo em um papel mais crítico ajuda a puxar um tipo de atenção que um prompt neutro não consegue.

O Few-shot foi o único que não compilou, e o motivo mostra um risco real dessa técnica. O modelo seguiu o formato do exemplo com tanta força que usou a struct do exemplo (`GildedRose{Items: items}`) em vez da função que realmente estava no código fornecido (`UpdateQuality(items []*Item)`), além de cometer um erro de sintaxe (`*testing.t` em vez de `*testing.T`). Ou seja, o modelo priorizou imitar a forma do exemplo em vez de prestar atenção no código real que foi passado. Esse é um ponto de atenção importante para quem for usar few-shot, o exemplo precisa estar bem alinhado com o contexto real, senão o modelo pode simplesmente copiar a estrutura errada.

Já o Spec-driven mostra que dar as regras prontas não garante uma resposta livre de erro, só muda onde o erro aparece. Os testes compilaram e tiveram boa cobertura, mas o texto explicativo do modelo ficou errado, ele disse que três dos testes deveriam falhar quando na verdade eles passam. Isso reforça que, mesmo quando a lógica de negócio é dada de bandeja, ainda é preciso rodar os testes para confirmar o que o modelo afirma, porque a explicação em texto pode não bater com o comportamento real do código.

De forma geral, dá para concluir que a técnica de prompt influencia tanto a corretude do resultado, se compila ou não, quanto a profundidade da cobertura de casos de borda. As técnicas que pedem para o modelo pensar antes de gerar o código, como o CoT e a Persona, geraram testes mais

confiáveis e explicações mais alinhadas com o código gerado. Já técnicas baseadas em imitar um formato, como o Few-shot, correm o risco de o modelo copiar a estrutura do exemplo mesmo quando ela não faz sentido para o problema real. E mesmo fornecer a especificação pronta, como no Spec-driven, não garante que a narrativa do modelo sobre os próprios testes esteja certa. No fim, nenhuma das cinco técnicas substitui o trabalho de compilar e rodar os testes para confirmar se o que foi gerado realmente funciona.